DBDOME Update

Deploy · Runscript · Signing · DB Upgrade

High-Level Architecture & Process

dbdome_update.exe — incremental updater (v2.0.0) for an existing DBDOME install; media auto-detected

Refreshes signed binaries + Grafana, then upgrades the database via enumerated, checksum-tracked SQL scripts

Preserves PostgreSQL, collected data and bin\.env — no PG/Oracle reinstall

Incremental updater — changes vs preserves

- **dbdome_update.exe (v2.0.0) — updates an EXISTING install from a local/removable package; auto-detects the media (scans mounted drives for <drive>:\dbdome, prefers removable, skips C:); --source override. Run as Administrator.**
- **Changes — appliance binaries (dbdome_service.exe + bin\ + _internal), reinstalls the Windows services, refreshes Grafana (grafana.db), and applies pending DB migrations.**
- **Preserves — the PostgreSQL install & data, all collected metrics/alerts/incidents, and bin\.env (real keys) are untouched.**
- **Not a re-install — no PostgreSQL/ODBC/Oracle-client install (setup-only). The DB backup + ODBC msi ride along ONLY as a fresh-DB fallback (base_install runs only if the catalog is empty).**
- **Signed & trusted — binaries are Authenticode-signed; the updater re-imports the public dbdome_cert.cer into the machine trust store.**
- **DB upgrade = enumerated SQL + ledger — the database is upgraded by applying the ordered postgres\install\NNNN_*.sql set; the checksum ledger meta.schema_migrations applies only what is new or changed (idempotent).**

Update package composition

dbdome_update.exe
updater (update.py)

dbdome\ payload

applied onto the existing install

bin\

dbdome_dbanalytics.exe · dbdome_service.exe · _internal · certification\dbdome_cert.cer · nssm.exe

conf\ + data\grafana.db

Grafana refresh (dashboards/config); dbdomeg served via NSSM

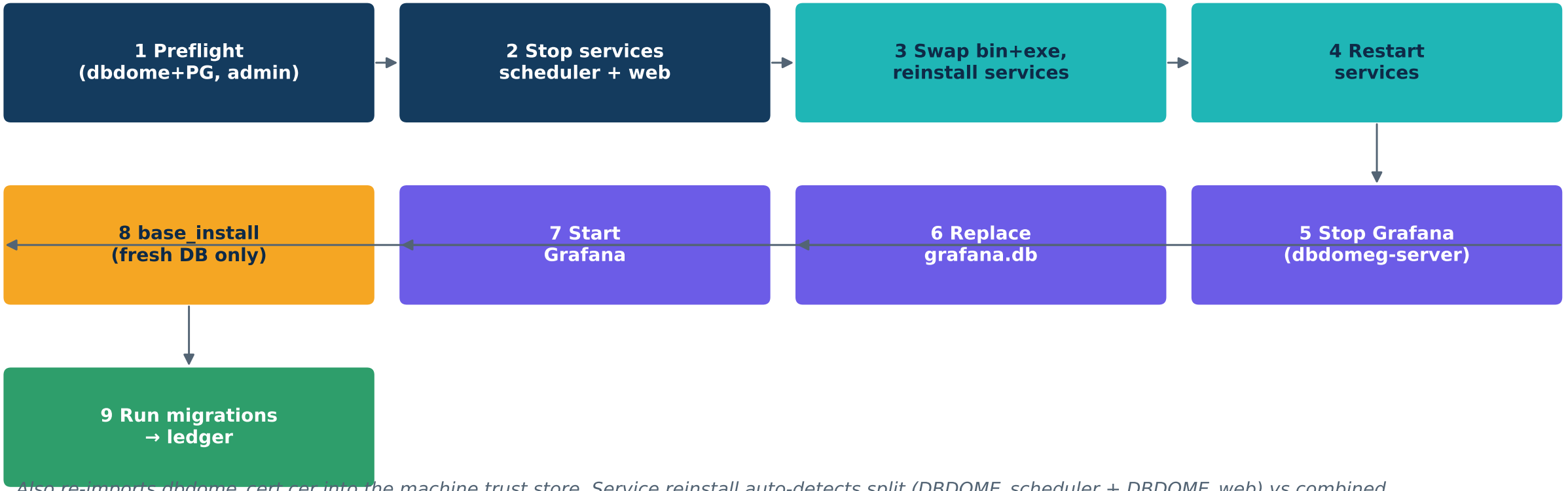
postgres\install\

base_install\ + 302 enumerated migrations 0001–7350 (the DB upgrade set)

fresh-DB fallback only

dbanalytics_install.backup · msodbcsql.msi (NO PostgreSQL-18 / Oracle client — setup-only)

Update flow — dbdome_update.exe



Also re-imports dbdome_cert.cer into the machine trust store. Service reinstall auto-detects split (DBDOME_scheduler + DBDOME_web) vs combined (DBDOME_dbanalytics). Steps 8–9 are the database upgrade.

Deploy process

- **On the target (update.py) — stop DBDOME_scheduler + DBDOME_web → replace dbdome_service.exe + bin\ + _internal → reinstall & restart services → stop dbdomeg-server → replace data\grafana.db → start Grafana.**
- **Service model auto-detected — reinstall picks split (DBDOME_scheduler + DBDOME_web) or combined (DBDOME_dbanalytics) from the NEW exe's capability; combined builds reject --service.**
- **Preserved — bin\.env (PG creds, DBDOME_SECRET_KEY, ORG_IP) is never overwritten; PostgreSQL and its data stay in place.**
- **Build-side staging — deploy_all_three.ps1 copies the freshly-built, SIGNED dist_rd bundle into C:\installs\dbdome_update\dbdome\bin (no services); that payload is packaged into dbdome_update.exe and shipped on media.**
- **Trust — the updater re-imports the shipped public .cer so swapped signed binaries run without an “Unknown Publisher” prompt.**

Signature — signing & trust on update

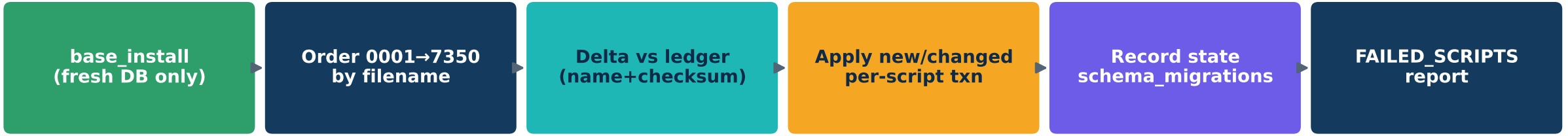
- **Signed at build — sign_dbdome.ps1 Authenticode-signs the dist_rd exes, the dbdome_update (and dbdome_setup) payload exes, and the installer executables (dbdome_update.exe / dbdome_setup.exe) with dbdome_cert.pfx — SHA-256 + RFC3161 trusted timestamp.**
- **Not re-signed — third-party binaries (grafana/dbdomeg, nssm) keep their vendors' signatures.**
- **Trust re-imported on update — update.py imports the public dbdome_cert.cer (shipped in bin\certification) into the machine trust stores every run, so refreshed signed binaries are trusted.**
- **Timestamped — the RFC3161 countersignature keeps signatures valid after the signing cert expires.**
- **Verify — signtool verify /pa confirms the chain; the live exes become signed simply by deploying the signed payload.**

Runscript engine — sql_script_runner.py

- **Ordering — all *.sql applied in strict case-insensitive FILENAME order; a zero-padded sequence prefix (0001_ … 7350_) is the sequence.**
- **Delta by checksum — each script is SHA-256 checksummed (line-ending-insensitive); runs only when its (filename, checksum) hasn’t succeeded before. New → apply; edited CREATE OR REPLACE → re-apply; unchanged → skip.**
- **Ledger = the version — meta.schema_migrations (filename, checksum, applied_at, execution_ms, success, error); the applied set IS the DB version — no explicit version number to track.**
- **Concurrency & atomicity — pg_advisory_lock(727274) serialises runners (each service startup calls it); each script + its ledger row commit in one transaction.**
- **Fail-open — a failing script is logged + recorded, then the run CONTINUES; surfaced as FAILED_SCRIPTS by the updater.**
- **Sources (additive, first-wins) — SQL_SCRIPTS_DIR → ..\postgres\install (shipped set) → exe\sql_scripts → bundled _internal\sql_scripts → repo. Payload + bundled exe feed one ledger; also re-run daily by the service.**

ENUMERATED SCRIPTS DRIVE IT

Database upgrade process



- **base_install — sql_scripts\base_install bootstraps schemas + tables on a FRESH/empty catalog only; on an existing install it is a no-op, so the same package installs or upgrades.**
- **Upgrade = apply the delta — the updater runs the whole ordered set, but the ledger makes it apply only scripts that are new or whose content changed since this box's last run. Upgrading from any prior version just works — ship the full set, the ledger decides.**
- **Deterministic & resumable — strict filename order + advisory lock + per-script atomic commit; a re-run after a failure retries only the unresolved scripts.**
- **Observability — every script lands as applied / re-applied / skipped / error in meta.schema_migrations; the updater prints a FAILED_SCRIPTS summary.**

302 MIGRATIONS (+ BASE_INSTALL)

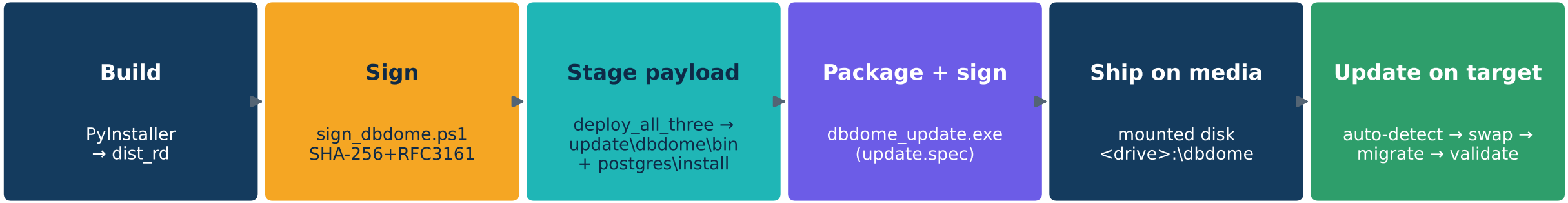
Enumerated SQL script bands

Band	#	Purpose	Representative scripts
base_install	—	fresh-DB bootstrap: schemas, tables, FKs, views & functions	00_prereqs · 00_install_all_tables · 90_foreign_keys · 95_views_and_functions
0001–0999	9	meta ledger · gmmr partitioning · alert_log + indexes · resultset fns	0001_meta_schema_migrations · 0010_migrate_gmmr_partition · 0100_alerts_log · 0300_fn_get_root_cause_resultset
1000–2999	170	rootcause DETECTION VIEW catalog (per-RC, per-vendor views)	create_view_v_sec_sql_<family>_<NNN>_rcNN (acc/au/aud/authz/cfg/enc/qe/tx…)
5000–5999	35	custom metrics · server registration · multi-vendor detection fixes	5xxx custom_metrics / server registration / vendor dialect fixes
6000–6999	81	RBAC · content/masking hardening · incident lifecycle · exec-plan · audit · health	6550 incident lifecycle · 6690 RBAC hardening · 6280 parse_execution_plan · 63xx AUD rootcauses
7000–7399	7	ClickHouse · DAM scorecard · content encryption · partitioning	7280 DAM scorecard · 7300/7310 content encryption · 7340 alert_log partition · 7350 rc02 strip

Idempotency & worked examples

- **One-shot migrations (run exactly once) — structural/data changes whose checksum never changes, e.g. 7340_partition_alert_log (converts alerts.alert_log to a 2-level RANGE→HASH partitioned table). The ledger prevents any re-run.**
- **Re-applied on edit (CREATE OR REPLACE) — detection views (1000–2999 create_view_v_*) and functions: editing the SQL changes the checksum, so the next update re-applies just those; unchanged ones are skipped.**
- **Encryption migrations — 7300_rootcause_content_encryption / 7310_encrypt_detection_logic encrypt the detection catalog at rest via idempotent enc() — safe to re-run.**
- **Self-healing — 7350_rc02_strip_execution_plan_xml adapts to the current RC02 SQL and wraps it only if not already wrapped.**
- **Why it is safe — advisory-locked single runner + per-script transaction (script + ledger row commit together) + fail-and-continue; a partial/failed upgrade is simply resumed on the next run.**

End-to-end: build → ship → update



- **Binaries flow one way — build → sign → stage into the update payload → package → media → target. Customers only ever receive signed artifacts.**
- **Schema travels as files — new/changed NNNN_*.sql are dropped into postgres\install and shipped; the runscript + ledger reconcile them on the target. No rebuild to ship a detection/view change.**

Operational safety, rollback & key files

- **Update safety — preflight (dbdome + PostgreSQL present, admin); service reinstall auto-detects split/combined; grafana.db swap bracketed by stop/start; bin\.env preserved; FAILED_SCRIPTS summary at the end.**
- **DB-upgrade safety — base_install guarded to fresh DB; ledger delta (apply only new/changed); advisory-locked; per-script atomic transaction; fail-and-continue then resume on re-run.**
- **Signing safety — RFC3161 timestamp (survives cert expiry); third-party binaries never re-signed; .cer trust import scoped to the machine store.**
- **Key files — dbdome_update/update.py (9-step updater) · update.spec · processes/sql_script_runner.py · payload postgres\install\ (base_install + 302 migrations) · certification/{sign_dbdome.ps1, dbdome_cert.*} · deploy_all_three.ps1 (staging).**
- **Secrets — DB & PFX passwords live only in the operator scripts / host .env — never in the shipped catalog or this document.**